

Unidad III: Diccionario y expresión en Java

Clase 4 – Diccionario: clave, valor y diseño modular

Objetivo

Entender cómo expresar un TDA en Java manteniendo modularidad.

Tema

- TDA Diccionario (simple y múltiple).
- Interfaces e implementación en Java.
- Separación entre contrato y lógica interna

TDA.Conceptos y especificación

***Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.*

Diccionarios

El último tipo de dato abstracto que vamos a definir en esta unidad será el diccionario. La estructura de datos diccionario se caracteriza porque cada valor ingresa a la estructura asociado a una clave, y estas claves existen siempre que tengan valor asociado y son únicas. En el presente curso se van a analizar dos comportamientos diferentes de la estructura diccionario, el primero de ellos será en donde cada clave puede tener asociado un único valor, estructura que denominaremos DiccionarioSimple, y el segundo en donde cada clave tiene asociado un conjunto de valores, estructura que denominaremos DiccionarioMultiple.

2.3.5.1 Diccionario Simple

A continuación, vamos a listar las operaciones, junto a sus comportamientos, que nos permite especificar el TDA de un diccionario simple:

- **Agregar:** dada una clave y un valor, agrega al diccionario el valor quedando asociado a la clave. Si ya existe la misma clave con otro valor se sobrescribe el valor, dejando el nuevo ingresado. Para poder agregar un par clave/valor la estructura debe estar inicializada.
- **Eliminar:** dada una clave elimina el valor asociado a la clave, y por consiguiente la clave ya que el diccionario no puede contener claves sin valores asociados. Si la clave no existe no se hace nada. Se supone que el diccionario está inicializado.

- Recuperar: dada una clave devuelve el valor asociado a la clave. La clave dada debe pertenecer al diccionario. Se supone que el diccionario esta inicializado.
- Claves: devuelve el conjunto de todas las claves definidas en el diccionario. Se supone que el diccionario esta inicializado.
- InicializarDiccionario: permite inicializar la estructura del diccionario.

Es decir, el tipo de dato abstracto DiccionarioSimpleTDA nos quedaría de la siguiente manera:

```
public interface DiccionarioSimpleTDA {
    void InicializarDiccionario();
    // siempre que el diccionario esté inicializado
    void Agregar(int clave, int valor);
    // siempre que el diccionario esté inicializado
    void Eliminar(int clave);
    // siempre que el diccionario esté inicializado y la clave exista en el mismo
    int Recuperar(int clave);
    // siempre que el diccionario esté inicializado
    ConjuntoTDA Claves();
}
```

2.3.5.2 Diccionario Múltiple

El diccionario múltiple tiene como característica principal que cada clave del diccionario puede tener asociado un conjunto de valores.

A continuación, vamos a listar las operaciones, junto a sus comportamientos, que nos permite especificar el TDA de un diccionario múltiple:

- Agregar: dada una clave y un valor, agrega al diccionario el valor quedando asociado a la clave. Una misma clave puede tener asociada un conjunto de valores, pero esos valores no se pueden repetir. Para poder agregar un par clave/valor la estructura debe estar inicializada.
- Eliminar: dada una clave elimina todos los valores asociados a la clave, y por consiguiente la clave ya que el diccionario no puede contener claves sin valores asociados. Si la clave no existe no se hace nada. Se supone que el diccionario esta inicializado.
- EliminarValor: dada una clave y un valor se elimina el valor asociado a la clave, y en caso de que la clave o el valor no existan no se hace nada. Si al eliminar el valor, la clave no tiene otros valores asociados se debe eliminar la misma. Se supone que el diccionario esta inicializado.
- Recuperar: dada una clave devuelve el conjunto de valores asociados a la misma. Si la clave dada no pertenece al diccionario, se debe devolver un conjunto vacío. Se supone que el diccionario esta inicializado.
- Claves: devuelve el conjunto de todas las claves definidas en el diccionario. Se supone que el diccionario esta inicializado.

- InicializarDiccionario: permite inicializar la estructura del diccionario.

Es decir, el tipo de dato abstracto DiccionarioMultipleTDA nos quedaría de la siguiente manera:

```
public interface DiccionarioMultipleTDA {
    void InicializarDiccionario();
        // siempre que el diccionario esté inicializado
    void Agregar(int clave, int valor);
        // siempre que el diccionario esté inicializado
    void Eliminar(int clave);
        // siempre que el diccionario esté inicializado
    void EliminarValor(int clave, int valor);
        // siempre que el diccionario esté inicializado
    ConjuntoTDA Recuperar(int clave);
        // siempre que el diccionario esté inicializado
    ConjuntoTDA Claves();
}
```

Implementación

Implementación de Diccionario

A continuación, vamos a llevar a cabo la implementación de los TDA de diccionarios, a partir de la utilización de arreglos. En el caso del diccionarioSimple se emplea un único arreglo de elementos que se definen por una clave y un valor. mientras que en el diccionario Múltiple vamos a emplear un arreglo de elementos donde cada uno de estos contiene una clave y un arreglo de valores. Dado que las claves y valores de un diccionario no tienen que mantener un criterio de ordenamiento particular, cuando se elimina un elemento se puede colocar (al igual que en la implementación del conjunto) el último elemento que se encuentra en el arreglo en la posición del elemento a eliminar, y de esta manera evitar el desplazamiento de todos los elementos.

Diccionario Simple

```
public class DicSimpleA implements DiccionarioSimpleTDA {
```

```
    class Elemento{
        int clave;
        int valor;
    }
```

```

Elemento[] elementos;

int cant;

public void inicializarDiccionario() {
    cant = 0;
    elementos = new Elemento[100];
}

public void agregar(int clave, int valor) {
    int pos = Clave2Indice(clave);
    if (pos == -1) { // Si no está la clave, la agrega
        pos = cant;
        elementos[pos] = new Elemento();
        elementos[pos].clave = clave;
        cant++;
    }
    elementos[pos].valor = valor; // Actualizo valor
}

public void eliminar(int clave) {
    int pos = Clave2Indice(clave);
    if (pos != -1) {
        elementos[pos] = elementos[cant-1];
        cant--;
    }
}

public int recuperar(int clave) {
    int pos = Clave2Indice(clave);
    return elementos[pos].valor;
}

```

```

public ConjuntoTA claves() {
    ConjuntoTA c = new ConjuntoTA();
    c.inicializarConjunto();
    for(int i =0;i<cant;i++) {
        c.agregar(elementos[i].clave);
    }
    return c;
}

private int Clave2Indice(int clave) {
    int i=cant-1;
    while(i>=0 &&elementos[i].clave!=clave) {
        i--;
    }
    return i;
}
}

```

Diccionario Múltiple

```

public class DicMultipleA implements DiccionarioMultipleTDA {

    class Elemento{
        int clave;
        int[] valores;
        int cantValores;
    }

    Elemento[] elementos;
    int cantClaves;

    public void inicializarDiccionario() {
        elementos = new Elemento[100];
        cantClaves = 0;
    }
}

```

```

public void agregar(int clave, int valor) {
    int posC = Clave2Indice(clave);
    if(posC == -1) {
        posC = cantClaves;
        elementos[posC] = new Elemento();
        elementos[posC].clave = clave;
        elementos[posC].cantValores = 0;
        elementos[posC].valores = new int[100];
        cantClaves++;
    }
    Elemento e = elementos[posC];
    int posV = Valor2Indice(e, valor);
    if(posV == -1) {
        e.valores[e.cantValores] = valor;
        e.cantValores++;
    }
}

private int Valor2Indice(Elemento e, int valor) {
    int i = e.cantValores-1;
    while(i >= 0 && e.valores[i] != valor) {
        i--;
    }
    return i;
}

public void eliminar(int clave) {
    int pos = Clave2Indice(clave);
    if(pos != -1) {
        elementos[pos] = elementos[cantClaves-1];
        cantClaves--;
    }
}

```

```
public void eliminarValor(int clave, int valor) {
    int posC = Clave2Indice(clave);
    if (posC!=-1) {
        Elemento e = elementos[posC];
        int posV = Valor2Indice(e, valor);
        if (posV !=-1) {
            e.valores[posV] = e.valores[e.cantValores -1];
            e.cantValores--;
            if (e.cantValores==0) {
                eliminar(clave);
            }
        }
    }
}
```

```
public ConjuntoTA recuperar(int clave) {
    ConjuntoTA c=new ConjuntoTA();
    c.inicializarConjunto();
    int pos = Clave2Indice(clave);
    if (pos!=-1) {
        Elemento e= elementos[pos];
        for(int i=0; i<e.cantValores; i++){
            c.agregar(e.valores[i]);
        }
    }
    return c;
}
```

```

public ConjuntoTA claves() {
    ConjuntoTA c=new ConjuntoTA();
    c.inicializarConjunto();
    for( int i=0; i<cantClaves; i++){
        c.agregar(elementos[i].clave);
    }
    return c;
}

private int Clave2Indice(int clave) {
    int i = cantClaves-1;
    while(i>=0 && elementos[i].clave!=clave) {
        i--;
    }
    return i;
}
}

```

3.2 Análisis de Costos

Cuando queremos analizar o comparar diferentes implementaciones necesitamos algún parámetro o variables en la cual basar esa comparación, esos parámetros pueden ser:

- Cantidad de operaciones llevadas a cabo o utilizadas
- Cantidad de memoria utilizada

En el primer caso hablamos de análisis de complejidad temporal, y en el segundo de complejidad de espacio. En este curso haremos el análisis de complejidad temporal, y para ello realizaremos un análisis simplificado, sin entrar en conceptos demostrativos de complejidad temporal.

Para el análisis de costos de complejidad temporal se consideran los siguientes tres criterios a través de los cuales clasificaremos el costo de ejecución de un método:

- Costo constante (C): el costo no depende de la cantidad de elementos de entrada. Es el mejor de los costos posibles.
- Costo lineal (L): el costo depende linealmente de la cantidad de elementos de entrada, es decir que por cada elemento de entrada hay una cantidad constante de operaciones que se ejecutan. Es un costo bueno, aunque siempre depende del contexto.

- Costo polinómico (P): el costo es mayor a lineal. Un ejemplo de esta categoría sería el costo cuadrático. Es un costo no tan bueno, pero la interpretación depende del contexto.

A continuación, se muestran la resolución de los ejercicios 7, 8 y 9 del TP1.

Pila	PilaTI	PilaTF
InicializarPila	C	C
Apilar	L	C
Desapilar	L	C
Tope	C	C
PilaVacía	C	C

Cola	ColaPU	ColaPI
InicializarCola	C	C
Acolar	L	C
Desacolar	C	L
Primero	C	C
ColaVacía	C	C

Cola con Prioridad	ColaPrioridadDA	ColaPrioridadAO
InicializarCola	C	C
AcolarPrioridad	L (2 n)	L (2 n)
Desacolar	C	C
Primero	C	C
ColaVacía	C	C
Prioridad	C	C

Conjunto	Conjunto
InicializarConjunto	C
Agregar	L
Sacar	L
Pertenece	L
Elegir	C

Diccionarios	DiccionarioSimpleA	DiccionarioMutipleA
InicializarDiccionario	C	C
Agregar	L	L
Eliminar	L	L
EliminarValor	-	L
Recuperar	L	L
Claves	L	L